

PyTorch 2.0 TTNN: Интеграция бэкенда Tenstorrent через PrivateUse1

Обзор проекта и его целей

PyTorch 2.0 TTNN – это форк PyTorch, добавляющий поддержку ускорителей Tenstorrent с помощью собственного бэкенда **TT-NN**. Проект позволяет запускать PyTorch-модели на аппаратуре Tenstorrent как в обычном eager-режиме, так и через компиляцию графа. В репозитории представлены оба подхода:

- **Eager-режим:** достаточно открыть устройство Tenstorrent и перевести модель на него, после чего вызовы `forward` выполняются напрямую на ускорителе ¹. Например, в коде:

```
import torch, torch_ttnn, ttnn
model = YourModel()
device = ttnn.open_device(device_id=0) # открыть девайс
Tenstorrent
model.to(torch_ttnn.ttnn_device_as_torch_device(device)) # переместить
модель на устройство TTNN
output = model(input_data) # выполнение на устройстве
```

Здесь `torch_ttnn.ttnn_device_as_torch_device(device)` возвращает объект `torch.device` типа *PrivateUse1*, соответствующий открытому ускорителю Tenstorrent ².

- **Режим компиляции (TorchDynamo)** – **рекомендуемый** для максимальной производительности ³. Модель оборачивается через `torch.compile` с бэкендом `torch_ttnn.backend`, что заставляет PyTorch провести захват и компиляцию графа под ускоритель Tenstorrent. Например:

```
device = ttnn.open_mesh_device(ttnn.MeshShape(1, 2)) # открыть mesh
(например, 1x2 чипа)
option = torch_ttnn.TorchTtnnOption(device=device, data_parallel=2)
model = torch.compile(model, backend=torch_ttnn.backend, options=option)
output = model(input_data)
```

В этом режиме backend TTNN сам компилирует и сливает операции, достигая большей скорости, чем пошаговый eager ⁴.

Цель пользователя – подробнее разобраться во **внутреннем устройстве PyTorch на C++**, особенно в механизме интеграции нового устройства через PrivateUse1, и понять, как расширен PyTorch в проекте TTNN. В дальнейшем планируется **улучшить поддержку eager-режима** (сделать её более полной, быстрой) для данного бэкенда. Для этого нужно понять текущее состояние интеграции TTNN, что и как уже реализовано, и что следует добавить или изменить. Далее мы рассмотрим механизм PrivateUse1 в PyTorch, структуру решения TTNN и пути улучшения eager-режима, приводя низкоуровневые детали и простые примеры.

Механизм PrivateUse1 в PyTorch

PrivateUse1 – это зарезервированный **DispatchKey** (и соответствующий Autograd ключ) в PyTorch, предназначенный для внештатной интеграции нового бэкенда без изменения ядра PyTorch ⁵. До PyTorch 2.0 в ядре были зарезервированы три таких пары ключей (PrivateUse1/AutogradPrivateUse1, PrivateUse2, PrivateUse3) для экспериментальных бэкендов. Идея состоит в том, что внешние команды могут использовать эти ключи для регистрации своих устройств и операций. Это гораздо удобнее, чем добавлять новый ключ во встроенное перечисление DeviceType (что требует модификации множества файлов в PyTorch и ограничено 64 возможными ключами) ⁶. Вместо этого, воспользовавшись PrivateUse1, можно быстро прототипировать поддержку нового аппаратного ускорителя; при успешной интеграции в будущем PyTorch может выделить ему постоянный ключ (как это сделано для CUDA, XLA, MPS и др.) ⁵.

Однако механизм PrivateUse1 в версиях до PyTorch 2.1 был **неполным** – он охватывал не все аспекты инфраструктуры. К примеру, потребовались доработки, чтобы охватить **Storage** (хранение тензорных данных), интеграцию с **AMP** (Automatic Mixed Precision), распределённое обучение и другие модули для нового бэкенда ⁷. В PyTorch 2.1.0 сообщество провело серию улучшений, сделав PrivateUse1 более мощным и гибким. Теперь возможно сравнительно быстро и полноценно интегрировать новое устройство вне основного репозитория, минимизируя «лазейки» в поддержке ⁷.

Процесс интеграции бэкенда через PrivateUse1 включает несколько ключевых шагов (не все из них обязательны, но вместе они дают полный спектр возможностей) ⁸:

1. **Регистрация ядер операций (kernels).** Необходимо предоставить реализации операций PyTorch для нового устройства. Это делается через API диспетчера операторов: макрос `TORCH_LIBRARY_IMPL`. Вы указываете библиотеку (обычно `aten` для стандартных операторов) и ключ PrivateUse1, после чего регистрируете реализации. Например, чтобы переопределить сложение для нового бэкенда, регистрируется:

```
TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) {  
    m.impl("add.Tensor", TORCH_FN(my_private_add));  
    // ... другие оперы  
}
```

Тогда вызов `aten::add` на тензорах вашего устройства будет перенаправлен на функцию `my_private_add`. В официальном tutorialе показан пример регистрации `aten::add` для кастомного устройства ⁹. Аналогично регистрируются все нужные операторы: создание

тензоров (`empty`, `zeros` и т.д.), арифметические и тензорные операции, копирование и перемещение данных, специфичные функции и пр.

2. **Регистрация ядер автограда (Autograd).** PyTorch поддерживает **дифференцирование** с помощью либо встроенных формул градиента, либо разложения операций на более простые (*CompositeImplicitAutograd*). Если ваша операция является простым атомарным оператором, у PyTorch может быть дефолтная реализация backward (например, для `add` градиент тривиален – тождественное отображение – и не требует отдельного кода на устройстве). В таких случаях достаточно реализации для `PrivateUse1`: движок автограда использует «примитивы» и автоматом просуммирует градиенты ¹⁰. Но если операция **композиционная** (PyTorch не хранит для неё готовой формулы градиента, а рассчитывает через разбиение на более базовые операции), то при выполнении на новом бэкенде без специальных мер PyTorch попытается разложить её на части. Это может привести к ошибкам, если те части не реализованы на вашем устройстве. Решение – **зарегистрировать ядро и для AutogradPrivateUse1** (т.е. переопределить поведение автограда) ¹¹. Другими словами, нужно обеспечить и forward-, и backward-реализации на вашем бэкенде. Пример – функция потерь `aten::cross_entropy_loss`: она в PyTorch помечена как *CompositeImplicitAutograd* (нет отдельного backward, обычно вычисляется через softmax, логарифмы и т.д.). Если на бэкенде регистрировать только forward для `PrivateUse1`, PyTorch всё равно при backward попытается разложить её – и выдаст ошибку, что нет реализации для `AutogradPrivateUse1` (такое было показано на форуме) ¹² ¹¹. Правильно: зарегистрировать `cross_entropy_loss` и для `PrivateUse1`, и для `AutogradPrivateUse1` – тогда ваш код возьмёт на себя как прямой просчёт loss, так и градиент (либо за счёт собственной имплементации backward, либо предотвратит разложение и использует уже реализованные базовые операции на устройстве) ¹¹. В целом, для новых бэкендов стратегия такая: **примитивные ops** (имеющие готовый autograd у PyTorch) можно регистрировать только на `PrivateUse1` – autograd “подхватит” их (например, для `add` не нужен спец. backward) ¹⁰. **Составные ops** – либо реализовать полностью (forward+backward), либо позволить разложиться на более мелкие, но тогда эти мелкие операции тоже должны быть реализованы.

3. **Регистрация генератора (Random API).** Если требуется поддержка генерации случайных чисел на новом устройстве (например, `torch.randn(device=PrivateUse1)`), нужно реализовать генератор на стороне бэкенда. PyTorch позволяет динамически добавить генератор: достаточно унаследовать `c10::GeneratorImpl` для вашего устройства и реализовать методы (например, на основе генератора в API Tenstorrent), а затем зарегистрировать фабрику через макрос `REGISTER_GENERATOR_PRIVATEUSE1(my_generator_builder)` ¹³ ¹⁴. После этого вызовы вроде `torch.Generator(device='privateuseone')` или функции, полагающиеся на глобальный генератор устройства, будут знать о вашем Random Engine. В рамках TTNN это означает, что такие операции как `torch.rand` или слои Dropout могут использовать генератор Tenstorrent (если реализован) или, по крайней мере, не падать с ошибкой отсутствия генератора.

4. **DeviceGuard и управление контекстом устройства.** PyTorch имеет механизм `DeviceGuard` для управления текущим активным устройством и потоками (аналоги `cudaSetDevice`, контекстные менеджеры `with torch.cuda.device(...)` и т.п.). Для нового бэкенда можно зарегистрировать свою реализацию `DeviceGuardImplInterface`, которая знает, как

переключать контекст на ваше устройство (например, устанавливать какой чип/поток текущим) ¹⁵. Регистрация осуществляется через макрос `C10_REGISTER_GUARD_IMPL(PrivateUse1, MyGuardImpl)`. После этого PyTorch при выполнении операторов на вашем девайсе будет вызывать эти хуки – например, перед выполнением kernel может вызвать `guard.setDevice(device_index)` вашего бэкенда, а по завершении – восстановить предыдущий контекст. В итоге, внутри C++ движка правильно поддерживается текущий device типа PrivateUse1 (как и для CUDA или другого). В частности, это предотвращает ситуации, когда, скажем, два устройства вашего типа открыто, и нужно убедиться, что операция выполняется на том, которому принадлежит тензор.

5. Сериализация/десериализация. Если требуется сохранять и загружать модели с тензорами нового устройства, нужно позаботиться о поддержке этого в механизме `torch.save/torch.load` (сердце которого – `pickle + torch.serialization`). PyTorch 2.1 добавил возможность регистрировать функции сериализации для «метаданных бэкенда» (`BackendMeta`) ¹⁶ ¹⁷. Вы можете определить структуру, наследующую `BackendMeta`, чтобы хранить специфичную информацию (если нужно), и зарегистрировать две функции: одна заполняет словарь метаданных при сохранении тензора на вашем девайсе, другая восстанавливает (или валидирует) при загрузке. В простейшем случае, возможно, достаточно зарегистрировать имя устройства, чтобы PyTorch знал, как обозначить PrivateUse1-тензор в файле (например, `'privateuseone:0'`). В противном случае, при попытке сохранить модель с такими тензорами, PyTorch может выдать предупреждение или ошибку, что устройство неизвестно. TTNN, вероятно, добавляет метаданные, чтобы при загрузке можно было автоматически понять, что тензор относится к устройству Tenstorrent. (Либо же обходной путь – при сохранении переводить всё на CPU, что не всегда удобно.)

6. Прочие модули. Помимо вышеперечисленного, поддержка нового бэкенда может быть расширена и в других подсистемах. Например, **распределённые операции** (коллективы для многопроцессового или межзвучного обучения) – PyTorch позволяет регистрировать их реализацию для PrivateUse1, хотя это более специфично (см. обсуждения о `c10d` с PrivateUse1). Другой пример – интеграция с системой таймеров для бенчмарков, поддержка sparsity (разреженных тензоров) и т.д. Эти аспекты пока менее формализованы, но в будущем тоже могут быть добавлены для PrivateUse1 ¹⁸. В целом, перечисленные 5 пунктов покрывают основную функциональность, необходимую чтобы тензоры нового устройства были «первоклассными гражданами» в PyTorch.

Важно отметить, что **PyTorch 2.1** также предоставляет возможности улучшения удобства использования бэкендов PrivateUse1. В частности, можно **зарегистрировать Python-модуль устройства**, чтобы дать ему человекочитаемое имя и функции управления. Например, вы можете вызвать `torch._register_device_module("myhw", module)` и реализовать в нём API (функции вроде `is_available()`, управление AMP и др.) ¹⁹ ²⁰. Это также позволяет переопределить строковое имя устройства: вместо `'privateuseone'` тензоры могут показываться как `'myhw'` – то есть, по сути, alias для лучшего опыта пользователя. Мы вернёмся к этому в контексте TTNN.

Реализация бэкенда TTNN (Tenstorrent) в проекте

В репозитории `pytorch2.0_ttnn` реализованы вышеописанные механизмы для поддержки **Tenstorrent TT-NN** ускорителей. Рассмотрим, как он устроен, на основе открытых данных и типичных шагов интеграции:

- **Девайс и устройство в PyTorch:** Tenstorrent предоставляет API (TT Metalium, TT NN) для работы с своим чипом. В TTNN-репозитории есть модуль `ttnn` (скорее всего Python-обёртка для C++ библиотеки), и Python-пакет `torch_ttnn`, который связывает это с PyTorch. Когда вы вызываете `ttnn.open_device(0)`, происходит открытие устройства (например, карточки Wormhole) через драйвер Tenstorrent; возвращается дескриптор/объект устройства. Функция `torch_ttnn.ttnn_device_as_torch_device(device)` берёт этот дескриптор и создает соответствующий `torch.device`. Внутри она, вероятно, делает что-то вроде: **регистрирует** (или подтверждает) тип `PrivateUse1` и берёт **индекс** 0, после чего возвращает `torch.device("privateuseone", 0)`. В самом деле, PyTorch уже знает строку `"privateuseone"` для `DeviceType::PrivateUse1`, так что, возможно, библиотека просто возвращает `torch.device("privateuseone", device_idx)`. Вероятно, разработчики TTNN могли зарегистрировать и **алиас** типа, например `"tenstorrent"/"ttnn"`, но судя по их примерам, они используют функцию для получения объекта устройства, скрывая реализацию. Таким образом, **тензор, перемещённый на TTNN-девайс, внутри PyTorch помечается как `device=PrivateUse1`**.

- **Регистрация операторов TTNN:** Проект предоставляет широкий набор операций, оптимизированных для архитектуры Tenstorrent. В официальной документации TT-NN перечислены поддерживаемые функциональности: от базовых операций создания тензоров (`ttnn.zeros`, `ttnn.ones` и т.п.) до матричных множителей (`ttnn.matmul`, `ttnn.linear`) и множества поэлементных функций (`sin`, `cos`, `clamp`, и даже экспериментальные, вроде `ttnn.experimental.dropout`)²¹ ²². Каждая такая функция на самом деле осуществляет вызов в низкоуровневый runtime (Metalium) для исполнения на чипе. Для интеграции с PyTorch, TTNN-регистрации, скорее всего, используют подход с `TORCH_LIBRARY_IMPL(aten, PrivateUse1, ...)`. Например, они должны были зарегистрировать у диспетчера ядро `aten::add.Tensor` для `PrivateUse1`, которое внутри вызывает соответствующую функцию TTNN (сложение двух тензоров на чипе). То же для других операций: `aten::matmul` -> TTNN матмул, `aten::conv2d` -> возможно, композитно разложен на TTNN `matmul` + иное, либо собственный kernel, и т.д. Благодаря этой регистрации, когда модель находится на устройстве TTNN, стандартные PyTorch операции (**API остается тем же!**) вызывают ядра Tenstorrent. Пользователь в Python просто пишет, например, `y = x * 2` или `y = torch.nn.functional.linear(x, w)` – если `x`, `w` на `PrivateUse1`, сработают TTNN-версии. Если какая-то операция не была зарегистрирована, PyTorch выдаст ошибку вида *"Could not run ... from the 'PrivateUse1' backend"*, указывая, что для данного backend отсутствует kernel (эти сообщения перечисляют, для каких backend доступна операция)¹². В TTNN стремятся покрыть как можно больше операторов, но, возможно, на PyTorch 2.0 часть нужно было дописать. (Мы обсудим ниже стратегию fallback на CPU, которую можно использовать для пропущенных операций.)

- **Копирование данных и создание тензоров:** Ключевая часть – уметь **создавать тензоры на устройстве** и **перемещать** их между хостом (CPU) и ускорителем. TTNN реализует **преобразование torch.Tensor <-> ttnn.Tensor**. В их API есть функции `ttnn.from_torch(torch_tensor)` и `ttnn.to_torch(ttnn_tensor)`^{23 24}. Когда вы делаете `model.to(torch_ttnn.ttnn_device_as_torch_device(device))`, под капотом происходит примерно следующее: для каждого параметра/тензора модели PyTorch вызывает оператор `copy/to`. Этот оператор для пары `src` (CPU) и `dst` (PrivateUse1) должен быть зарегистрирован. Вероятнее всего, TTNN добавил реализацию для `aten::to` (или `_to_copy`) и/или `aten::copy_` на комбинации (CPU -> PrivateUse1 и PrivateUse1 -> CPU). Таким образом, вызов `.to(device)` на тензоре вызывает TTNN: библиотека аллоцирует память на устройстве Tenstorrent, копирует содержимое CPU-тензора через PCIe в память ускорителя (DRAM или SRAM, в зависимости от реализации), и возвращает новый `TensorImpl`, привязанный к PrivateUse1, у которого `data_ptr` указывает на буфер в памяти устройства. Аналогично, при обратном копировании (PrivateUse1 -> CPU) – читает данные с ускорителя обратно в память RAM. Без этих методов невозможно было бы ни инициализировать тензоры на устройстве, ни получить результаты обратно. Судя по тому, что `ttnn.open_device` возвращает объект, который затем используется для `.to(device)` – TTNN, вероятно, хранит «состояние» устройства (например, `handle` и контекст) внутри себя, а в PyTorch регистрируется глобальный указатель на текущий активный PrivateUse1 девайс (подобно тому, как `cudaMemcpy` имплицитно знает, на какой GPU `copy`). **Аллокация памяти** для PrivateUse1-тензоров, скорее всего, решается через *custom allocator*. PyTorch позволяет привязать аллокатор к определённому `DeviceType`. TTNN, вероятно, зарегистрировал в `c10::AllocatorRegistry` свой аллокатор для PrivateUse1, который внутри вызывает `ttnn.allocate(size)` – т.е. запрашивает буфер на устройстве (в DRAM Tenstorrent-чипа). Если PyTorch 2.0 не имел для этого удобного API, возможно, разработчики просто создали `StorageImpl` с кастомным `deleter`. В любом случае, раз модель успешно создаёт параметры на устройстве, значит, `Storage/Allocator` для PrivateUse1 функционирует (это часть «Storage» поддержки, о которой говорилось в обновлениях PyTorch 2.1).

- **DeviceGuard и текущий девайс:** Для корректной работы, TTNN реализует логику для `DeviceGuardImplInterface` – скорее всего, структуру **TorchTtnnDeviceGuard** или подобную. При импорте модуля `torch_ttnn` вероятно вызывается регистрация: `C10_REGISTER_GUARD_IMPL(PrivateUse1, TorchTtnnGuard)`. Этот `guard` обеспечит, например, что при последовательном вызове нескольких операций на PrivateUse1 PyTorch может **установить «текущий» Tenstorrent-девайс**. На практике, если у вас один ускоритель, это может быть тривиально `poor` (он всегда `device 0`). Но если, допустим, система поддерживает 2 карты Tenstorrent, PyTorch должен знать, какой из них активен – аналогично `torch.cuda.set_device(...)`. Guard реализует методы вроде `setDevice`, `getDevice`, `exchangeDevice`, `uncheckedSetDevice` и др. Внутри он, возможно, вызывает `ttnn.set_active_device(index)` – гипотетический вызов в библиотеке Tenstorrent, который направляет последующие команды на выбранный чип. Это важно, если, скажем, асинхронные операции или распределение на несколько устройств используются. В `eager`-режиме с одним девайсом это менее заметно, но для полноты интеграции – реализовано.

- **Autograd (поддержка обратного прохода):** Для тренировки моделей в `eager`-режиме критично, чтобы **градиенты могли распространяться на устройстве**. Как упоминалось, PyTorch либо использует зарегистрированные `AutogradPrivateUse1` kernels, либо пытается

выполнить backward через стандартные операции. Например, градиент по `add` – это просто `ones_like`, который PyTorch выполнит на том же устройстве (должен быть реализован `aten::ones_like` на PrivateUse1, или хотя бы `aten::empty_like` + fill). Градиент по матмул: выражается через еще один матмул (с транспонированием). Если `aten::matmul` есть, и `aten::transpose` (или `permute`) на PrivateUse1 есть, то автоград сможет посчитать градиенты средствами самого бэкэнда. Но если какого-то шага нет – будет ошибка. Поэтому **полноценная поддержка тренировки** требует, чтобы либо все необходимые для backward операции тоже имелись, либо для них опять-таки предусмотрен fallback. В форуме PyTorch-разработчики советовали: для составных ops регистрировать **и PrivateUse1, и AutogradPrivateUse1** реализации ¹¹, чтобы переопределить не только forward, но и отключить дефолтное разложение backward. Например, TTNN может реализовывать свою fused backward-pass для cross_entropy или dropout. Некоторые сложные функции могут быть помечены как *CompositeImplicitAutograd*, и проще реализовать их целиком, чем поддерживать каждую мелкую операцию. Судя по представленному списку поддерживаемых моделей в README проекта, TTNN покрывает большой набор операций (в таблице указано количество “Torch Ops Remain (Unique Ops) = 0” для многих моделей – т.е. после компиляции все разбито на TTNN-операции) ²⁵. В eager-режиме, вероятно, достижение покрытия 100% сложнее, но к этому нужно стремиться.

- **Механизм резервного пути (fallback) на CPU:** Чтобы **обеспечить работу даже если какой-то оператор не реализован на устройстве**, PyTorch предоставляет полезную возможность – зарегистрировать *fallback* для всего набора PrivateUse1. Это делается через `TORCH_LIBRARY_IMPL(_, PrivateUse1, m)` – на глобальном пространстве (`_`) – с вызовом `m.fallback(...)`. В tutorialе, например, показано:

```
m.fallback(torch::CppType::makeFromBoxedFunction<cpu_fallback>());
```

где `cpu_fallback` внутри просто перенаправляет выполнение операции на CPU, используя уже существующие ядра ²⁶. Иначе говоря, если PyTorch не найдёт конкретного kernel под ключом PrivateUse1, он вызовет этот fallback. В fallback-функции можно:

- Снять тензоры с вашего устройства (скопировать на CPU)
- Вызвать оператор на CPU (через `at::native::cpu_fallback(op, stack)`, как в примере ²⁶, который автоматизирует этот процесс)
- Затем при необходимости вернуть результаты обратно на устройство (PyTorch `cpu_fallback` сам управляет стеком операций и возвращает тензор на исходный backend, если нужно).

Таким образом, **все не реализованные операции будут выполняться на CPU** автоматически. Цена – накладные расходы на копирование данных туда-обратно, потенциально большие, но модель будет не падать, а работать корректно. Для **отладки и экспериментов** это крайне полезно. Например, **на машине без физического ускорителя** вы могли бы протестировать PrivateUse1-бэкэнд с включённым fallback: PyTorch будет думать, что тензоры на некоем устройстве `privateuseone`, и пытаться звать TTNN. Реализовано далеко не всё? – он будет откатываться на CPU. Вы даже можете запустить модель целиком: она просто фактически посчитается на CPU, но пройдёт по всем путям кода интеграции. Это поможет убедиться, что цепочка от **Python->Dispatcher->TTNN/CPU fallback->...** работает. Неизвестно, включён ли такой fallback по умолчанию в TTNN

(вероятно нет, или он может быть отключаемым опционально). Но при разработке нового бэкенда можно на время **включить CPU fallback**, чтобы быстро получить «полную» функциональность на этапе становления. Затем, постепенно, fallback заменяется на реальные реализации для ускорителя.

В контексте TTNN, если ваша цель – «поддержка eager везде», имеет смысл **реализовать такой резервный путь**. Тогда даже редкие или сложные операции (скажем, какие-нибудь интерполяции или операции из сторонних библиотек) не будут сразу приводить к исключению – они выполнятся на CPU. Пользователь может получить корректный результат, пусть и с предупреждением о низкой производительности.

- **Alias и отображение устройства:** Ещё один момент – **удобное имя устройства**. По умолчанию, как мы обсуждали, PrivateUse1 в PyTorch называется "privateuseone". TTNN скрывает это под функцией `ttnn_device_as_torch_device`, но можно пойти дальше: зарегистрировать модуль `torch.devices.ttnn` (условно) и определить alias. PyTorch 2.x позволяет, например, чтобы `torch.device("ttnn:0")` распознавался. Если TTNN этим воспользовались, то внутри `torch_ttnn` могло быть что-то вроде:

```
torch._register_device_module('ttnn', torch_ttnn)
```

и определены функции `torch_ttnn.is_available`, `torch_ttnn.device_type()` и пр. Тогда в `torch.save` тоже могло бы сохраняться как `<Tensor... device='ttnn:0'>`. Не имея прямого доступа к их исходникам (репозиторий – форк всего PyTorch, и код интеграции распределён), мы не утверждаем наверняка, но **PyTorch предоставляет такую возможность переименования** ²⁰. В любом случае, на уровне C++ DeviceType останется PrivateUse1, просто при отображении для пользователя идёт дружественное имя.

- **Пример сквозного использования:** Обобщим типичный сценарий работы TTNN в eager-режиме на простом примере – **векторное сложение**:

- Пользователь на Python создаёт тензор: `x = torch.tensor([1,2,3])` – по умолчанию на CPU.
- Открывает устройство: `dev = ttnn.open_device(0)`.
- Копирует тензор: `x_tt = x.to(torch_ttnn.ttnn_device_as_torch_device(dev))`. Это вызывает `Tensor::to(...)` внутри PyTorch C++, который через диспетчер найдёт kernel `aten::to.other` для (CPU->PrivateUse1). TTNN зарегистрировал его, и kernel выполняет: **аллоцирует буфер на Tenstorrent, копирует 3 элемента 1,2,3 туда**. Возвращается новый `Tensor` (Device=PrivateUse1, dtype=int64).
- Теперь `y_tt = x_tt + x_tt`. Python вызывает оператор `aten::add.Tensor` на two operands, оба PrivateUse1. Диспетчер ищет в таблице PrivateUse1 -> находит TTNN-реализацию (допустим, она вызывается `ttnn::add` из их API). Эта функция планирует и запускает на устройстве Tenstorrent kernel сложения (в аппаратных ядрах Tensix или RISC-V, в зависимости от реализации). Предположим, реализация асинхронная – она инициирует выполнение и сразу возвращает управление (или синхронно – тогда возвращает после завершения). Полученный результат – новый тензор PrivateUse1 с содержимым [2,4,6] на памяти ускорителя.

- `y = y_tt.to('cpu')` – опять копирование, но теперь исходный девайс PrivateUse1, целевой CPU. На этот раз вызывается kernel `aten::to.other` для (PrivateUse1->CPU), TTNN реализовал и его. Он запрашивает чтение данных [2,4,6] из памяти ускорителя в RAM и создаёт итоговый CPU Tensor с этими значениями.
- Весь процесс прозрачен для пользователя: он видит корректный результат тензора `y = [2, 4, 6]` на CPU. Внутри же PyTorch каждый вызов шёл через dispatcher с участием зарегистрированных TTNN функций.

Стоит отметить: если в шаге 4, вместо `+`, была бы операция, которую TTNN **не реализовал**, то при наличии fallback она автоматически: скопировала бы `x_tt` на CPU, выполнила бы операцию на CPU, и скопировала результат обратно на устройство (так, чтобы продолжить вычисления на нём). Без fallback – произошла бы ошибка NotImplemented для PrivateUse1.

Этот пример демонстрирует, как **PyTorch C++ internals взаимодействуют с пользовательским бэкэндом**: через систему **Dispatcher + DispatchKeys**. Для глубокого понимания можно обратиться к блогу Э. Янга о диспетчере PyTorch (Ezyang's blog), но в общих чертах: у каждого `at::Tensor` есть `DispatchKeySet` – битовый флаг, указывающий на тип (у TTNN-тензора будет флаг PrivateUse1). Когда вызывается оператор, PyTorch по этому ключу выбирает правильную реализацию (если она зарегистрирована). Autograd при вычислении градиентов также оперирует через dispatcher – сначала может попытаться вызвать специальный AutogradPrivateUse1 kernel, а если его нет и если он поддерживает композицию, разложит на последовательность других ops (которые опять же будут искаться по ключу). Поэтому интеграция требует думать в терминах «Какие ключи/ядра должны быть зарегистрированы для моей операции, чтобы все пути (forward + backward) покрыть?». TTNN именно этим и занимается внутри.

План улучшения поддержки eager-режима TTNN

Проанализировав устройство проекта, можно предложить ряд направлений, как сделать **eager-режим более полным, корректным и быстрым**:

- **Расширение покрытия операторов.** Для полной поддержки необходимо, чтобы все операции, используемые в целевых моделях, либо были реализованы на устройстве, либо имели приемлемый fallback. Практически, надо пройтись по моделям (в README TTNN уже перечислены протестированные архитектуры с указанием оставшихся Torch Ops) и убедиться, что для каждого встречающегося `aten::op` есть ядро PrivateUse1. Особое внимание к операциям, которые сейчас могут выполняться только при компиляции. Например, если в compile-режиме какая-то часть модели поддерживается посредством графовых трансформаций, то в eager она могла быть не реализована. Нужно реализовать эти пробелы. Это включает:
- **Элементарные арифметические и тензорные опции:** все `aten::add/sub/mul/div`, сравнения, логические, bitwise, агрегаты (`sum`, `mean`, `max` и т.д.), ресайпы (view, reshape, permute), конкатенации, срезы. Многие из них, вероятно, уже есть в TTNN (так как таблица показывает, что для рядовых моделей почти все ops покрыты) – но стоит проверить поддерживаются ли различные варианты схем (например, `add.out` версия, или broadcastable версии). Если чего-то нет – дописать с использованием TTNN API.
- **Операции linear algebra и свёртки:** матричные умножения, свёрточные слои, pooling, активации. TTNN наверняка реализует GEMM и conv (возможно, conv разлагается на matmul +

im2col, или компилируется). Для eager режима можно использовать ту же реализацию, что и в компиле (например, вызвать компиляцию маленького графа on-the-fly). Но более прямой путь – напрямую вызвать соответствующие TTNN primitive (в TTNN API могут быть `ttnn.conv2d` или, если нет, реализовать через несколько шагов TTNN).

- **Специфичные функции:** dropout, batchnorm, layernorm, сложные loss-функции, и т.д. Их поддержка может потребовать комбинирования операций. Например, BatchNorm можно выполнить формулой: вычислить mean, var (reduce ops), затем линейно трансформировать. Если reduce ops реализованы, можно собрать BN. Или написать единый kernel. Решение зависит от возможностей TTNN. Главное – гарантировать, что вызов `nn.BatchNorm(input_ttnn)` не упадёт. Опять же, composite ops: либо реализовать forward+backward, либо все составляющие.
- **Устранение проблем с autograd.** Даже после реализации всех forward-операций нужно проверить, проходят ли **обратные проходы** для обучения. Для этого можно написать небольшие тесты: взять тензор на TT устройстве, выполнить операцию, вызвать `backward()` и убедиться, что `tensor.grad` заполнен корректно. Если где-то вылетает `NotImplementedError: Could not run X from AutogradPrivateUse1`, значит PyTorch ожидал AutogradPrivateUse1 ядро. Например, так было с `cross_entropy_loss` в вопросе на форуме ¹². Решение: зарегистрировать *дополнительно* AutogradPrivateUse1 kernel (который либо вызывает ту же функцию, либо обёрнут через `torch::autograd::Function`). Либо – изменить тег операторов в PyTorch. В PyTorch многие сложные операции помечены как CompositeImplicitAutograd (т.е. нет ни Autograd, ни автограф-функции, autograd генерируется из разложения). Можно *принудительно пометить их как CompositeExplicitAutograd* (через core override), но лучше явно зарегистрировать. В частности, **функции потерь (cross_entropy, NLLLoss и др.), Dropout**, возможно **Softmax** – все они composite. Их backward либо разлагается (softmax backward вызывает снова softmax и умножения), либо требует реализации. Надо пройти по таким кейсам:
- Если опираемся на разложение: убедиться, что все составляющие есть на бэкенде (например, `softmax` backward делает вычисления с суммами и умножениями – они есть, ок).
- Если хотим ускорить или упростить: написать собственную autograd функцию (как в tutorialе показан CustomSelu ²⁷ ²⁸) и зарегистрировать её под AutogradPrivateUse1.
- Также проверить, что **Inplace-операции** корректно работают с autograd. PyTorch для in-place (например `x_tt += 1`) использует отдельный автограф ключ (InplaceOrView + AutogradPrivateUse1). В идеале, либо запрещать их (PyTorch может предупредить о задействовании неподдерживаемого view/inplace), либо реализовать. Например, для простоты можно сначала не разрешать сложные in-place на устройстве, скопировав их поведение из CPU (в худшем случае fallback справится).
- **Использование fallback для редких случаев.** Как обсуждалось, добавить **глобальный CPU-fallback** – очень полезно. Если в TTNN этого ещё нет, стоит внедрить:

```
TORCH_LIBRARY_IMPL(_, PrivateUse1, m) {
```

```
m.fallback(torch::CppType::makeFromBoxedFunction<at::native::cpu_fallback>());  
}
```

(в PyTorch >= 2.1 `at::native::cpu_fallback` доступен, в 2.0, возможно, придётся написать свой аналог, как в tutorialе ²⁶). С таким fallback, любой неохваченный оператор просто выполнится на CPU. Это значительно повышает *robustness* – модель не рухнет из-за одной мелочи. Конечно, на продакшн-сценариях, где важна скорость, следует стремиться покрыть всё своим ядром, но на этапе развития бэкенда fallback – спасение. К тому же, можно сделать его опциональным (флагом среды). Для домашнего эксперимента без доступа к «железу» – включить fallback и убедиться, что даже без ускорителя можно пройти кодом: `open_device` может эмулироваться (если его вызов без карты бросает исключение, можно предусмотреть режим симуляции, возвращающий фиктивный device, который всегда fallback-ится). Такой “toy” режим поможет **отлаживать PyTorch-интеграцию на обычном ПК (Ubuntu 24.04)**, не тратя время на прошивку и загрузку реального ускорителя.

- **Оптимизация производительности в eager.** Даже реализовав всё, нужно помнить, что **eager-режим, работая шаг за шагом, менее продуктивен для ускорителей**, чем режим графовой компиляции. Однако, есть способы улучшить:
- **Асинхронность.** Убедиться, что операции на устройстве выполняются асинхронно относительно CPU, когда это возможно. Например, для CUDA-тензоров PyTorch почти все операции ставятся в CUDA Stream и возвращают управление немедленно (лишь синхронизируясь при необходимости). Для Tenstorrent, если API позволяет запускать kernel асинхронно (что вероятно, через очереди команд), интеграция должна использовать это. Тогда Python сможет инициировать много мелких ops не дожидаясь каждой, а ожидание результатов будет происходить только при необходимости (например, при копировании на CPU или синхронизации). Это потребует в DeviceGuard/GuardImpl реализовать методы stream-менеджмента (эти части, возможно, тоже предусмотрены – PyTorch 2.x поддерживает Custom CUDA Streams для PrivateUse1). TTNN, возможно, уже делает асинхронно – но стоит проверить. Если пока всё синхронно, то добавить буферизацию команд и явную синхронизацию, улучшив суммарную производительность.
- **Минимизация копирований.** По возможности, избегать лишних переходов CPU<->Tenstorrent. Например, если какая-то промежуточная операция легче выполнить на хосте, но потом придётся вернуть на устройство – лучше сразу реализовать её на устройстве, чтобы не гонять данные. Fallback должен быть последней инстанцией. Тщательно продумать, какие ops можно **выполнить на CPU без потери производительности** (возможно, мелкие Tensor metadata ops – тип `x.shape` – и так на CPU делаются), а какие критично иметь на девайсе.
- **Совместимость с компиляцией.** Раз уж основной упор – compile, можно сделать так, чтобы eager-режим **повторно использовал часть compile-PIPELINE**. Например, если TorchInductor или torch-mlir применяются в TTNN, можно для некоторых сложных субграфов в eager вызывать JIT-компиляцию под капотом. Но это сложный путь и выходит за рамки обычного использования PyTorch. Обычно, eager – простой и понятный, compile – для макс. оптимизации. Поэтому возможно, достаточно честно предупредить: *“для production используйте compile, а eager – для отладки и быстрого прототипирования”*.

- **Automatic Mixed Precision (AMP) на устройстве.** Если Tenstorrent поддерживает упрощённую точность (например, BF16, FP8 и т.п.), хорошо бы интегрировать с механизмом автокаста PyTorch. Как мы упоминали, PyTorch позволяет регистрировать autocast-кernels под ключом `AutocastPrivateUse1`²⁹. TTNN мог бы указать, какие типы он поддерживает (например, FP16, BF16)¹⁹, и реализовать политики autocast (например, чтобы в режиме `amp_enabled` операции свёртки автоматически получали параметры в FP16 на входе). Это даст пользователям удобство: `with torch.autocast("privateuseone", dtype=torch.bfloat16): ...` – и модель частично в меньшей точности, экономия памяти/скорость. Если сейчас это не реализовано, можно добавить по аналогии с CUDA AMP (в PyTorch уже есть пример, как HPU или XLA добавляли). Это не напрямую про «eager vs compile», но про полноту поддержки устройства.
- **Поддержка multi-device и распределённости.** Возможно, менее актуально, но если предполагается использовать несколько карт Tenstorrent в одном узле, стоит проверить, что `torch.nn.DataParallel` или распределённый `DataParallel (c10d)` работают. Для этого нужно, чтобы операции типа `dist.all_reduce` были реализованы для `PrivateUse1` (или чтобы fallback их делал на CPU с копированием данных, что будет неэффективно). PyTorch 2.1 обещал улучшения в **distributed** для `PrivateUse1`, но деталей мало⁷. Если у Tenstorrent есть свой коммуникационный фреймворк (например, через Ethernet линк между картами), интегрировать его. Хотя, вероятно, это далеко вторично по сравнению с одиночной картой.
- **Обновление базы PyTorch.** Наконец, **рационально обновиться до PyTorch 2.1 или 2.2** (если доступно), так как многие вещи (в частности, fallback, сериализация, `torch._register_device_module` и др.) там улучшены. TTNN-форк основан на 2.0, поэтому какие-то части могли быть “костыльно” добавлены, что теперь есть штатно. Например, в 2.0 не было удобного API для регистрации backend name – в 2.1 появился. Смердживание этих изменений облегчит дальнейшую поддержку. Конечно, обновление – нетривиальная задача, но выигрыш в том, что **PrivateUse1 становится полноценным гражданином**: меньше шансов наткнуться на баг внутри PyTorch из-за `PrivateUse1` (в 2.0 такие баги были, судя по ишью на GitHub, например, про `.to(...)` для custom backend¹², многие из них исправлены к 2.1).

Подытоживая: чтобы **“было eager-везде, быстрее, круче”**, нужно довести интеграцию TTNN до уровня стандартных устройств. Следуя официальным рекомендациям PyTorch⁸, проект уже реализовал основное – но оставшиеся пробелы (операции или аспекты, не покрытые в 2.0) предстоит заполнить. Стоит воспользоваться механизмами `PrivateUse1` **динамически**: добавить недостающие ядра, зарегистрировать fallback на CPU²⁶, внедрить генераторы, AMP, и т.д. Благодаря примерам и тулзам PyTorch, это можно делать итеративно и локально.

Для разработчика, хорошо знакомого с Python API, эффективным путём будет: 1) начать с **простых экспериментов** на C++ уровне – например, написать минимальное расширение (как в официальном tutorialе), регистрирующее парочку операций на `PrivateUse1`, попробовать их вызвать в Python (Ubuntu 24.04 это позволит, нужно лишь собрать `extensão`)⁹. Это даст интуицию, как dispatcher выбирает реализации. 2) Далее, изучить существующий код TTNN (в `torch_ttnn` и `csrc` в репо Tenstorrent) – найти, где они вызывают `TORCH_LIBRARY_IMPL`, `REGISTER_GENERATOR_PRIVATEUSE1`, etc., посмотреть реализацию `open_device` (как он регистрирует device индекс). 3) Затем уже вносить изменения: например, добавить ядро для какого-то

оператора, собрать, протестировать на toy-модели (скажем, маленькая сеть, которую раньше **eager** не тянул, а теперь тянет).

Благодаря наличию fallback, вы можете проверять корректность постепенно: сначала включённый fallback обеспечит работу, а вы будете заменять его реальными оптимизированными операциями по одной.

Итак, проект **PyTorch 2.0 TTNN** – прекрасный пример глубокой интеграции нового устройства через PrivateUse1. Разобравшись с диспетчером, регистрацией операторов и устройством автограда, вы сможете уверенно вносить улучшения. Со временем, добавив все задуманные фишки (и, возможно, перейдя на актуальную версию PyTorch), eager-режим на Tenstorrent станет полностью прозрачным и эффективным для пользователей: модель будет работать на ускорителе просто по вызову `to(tt_device)`, сохраняя привычное поведение PyTorch, но на новой аппаратной платформе. Скомпилированный режим останется для максимальной производительности, но **понимание низкоуровневого слоя** позволит вам и в компиляторе внести улучшения при необходимости, а главное – обеспечит уверенность, что основа (eager) работает правильно.

Ссылки:

- Официальный tutorial по интеграции бэкэндов через PrivateUse1 (PyTorch Tutorials) – содержит примеры кода и описания шагов ³⁰ ²⁶ .
- Обсуждение на форуме PyTorch о регистрации составного оператора (cross_entropy) для кастомного бэкэнда – иллюстрирует нюансы AutogradPrivateUse1 ¹¹ ¹⁰ .
- Репозиторий Tenstorrent pytorch2.0_ttnn (GitHub) – README с примером использования TTNN в eager и compile режимах ¹ ⁴ .
- Документация Tenstorrent TT-NN – описывает доступные операции и API, которые соответствуют реализованным в PyTorch операторам ²³ ²⁴ .

¹ ² ³ ⁴ ²⁵ GitHub - tenstorrent/pytorch2.0_ttnn: ☆ TTNN Compiler for PyTorch 2 ☆ Enables running PyTorch models on Tenstorrent hardware using eager or compile path

https://github.com/tenstorrent/pytorch2.0_ttnn

⁵ ⁶ ⁷ ⁸ ⁹ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ Facilitating New Backend Integration by PrivateUse1 — PyTorch Tutorials 2.8.0+cu128 documentation

<https://docs.pytorch.org/tutorials/advanced/privateuseone.html>

¹⁰ ¹¹ ¹² How custom cross_entropy_loss register to PrivateUse1 and support backward - autograd - PyTorch Forums

<https://discuss.pytorch.org/t/how-custom-cross-entropy-loss-register-to-privateuse1-and-support-backward/178551>

²¹ ²² ²³ ²⁴ APIs — TT-NN documentation

<https://docs.tenstorrent.com/tt-metal/v0.56.0/ttnn/ttnn/api.html>